

Lecture Note: Phase 2 – Decision Making & Logic

1. Comparison & Logical Operators

To make decisions, Python evaluates conditions as Booleans (True or False).

- **Comparison Operators:**
 - `==` (Equal to), `!=` (Not equal to)
 - (Greater than), `<` (Less than)
 - `>=`, `<=` (Greater/Less than or equal to)
- **Logical Operators:**
 - `and`: True if **both** are true.
 - `or`: True if **at least one** is true.
 - `not`: Reverses the result (`not True` becomes False).

2. The `if` Statement

The most basic control structure. Code inside the block runs only if the condition is met.

Syntax:

```
Python
if condition:
    # Code to run
```

Note: Indentation (4 spaces) is mandatory in Python to define the block.

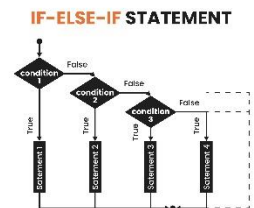
3. Branching with `elif` and `else`

Used to handle multiple mutually exclusive paths.

- **`elif` (else if):** Checks another condition if the previous ones were False.
- **`else`:** The "catch-all" that runs if no conditions were met.

Example:

```
Python
age = 18
if age >= 21:
    print("Full access")
elif age >= 18:
    print("Limited access")
else:
    print("Access denied")
```



4. Nested Conditions

You can place an `if` statement inside another `if` statement to create complex decision trees. Use this sparingly to maintain code readability.

5. Membership Operators

A quick way to check if a value exists within a sequence (like a string).

- **`in`:** Returns True if the value is found.
- **`not in`:** Returns True if the value is absent.
- *Example:* `"a" in "apple"` is True.

Common Pitfalls

1. **Assignment vs. Equality:** Using `=` (assign) instead of `==` (compare) inside an `if` statement will cause a Syntax Error.
2. **Missing Colons:** Forgetting the `:` at the end of `if`, `elif`, or `else` lines.
3. **Indentation Errors:** Inconsistent spacing will break the logic flow.